ESIGELEC
INGÉNIEUR.E.S GÉNÉRALISTES
SYSTÈMES INTELLIGENTS ET CONNECTÉS
ROUEN | POITIERS

Cryptography Fundamentals Lab 4 | Information Security Lab Report

AES | RSA-OAEP | SHA-2256 | RSA-PSS

Evidence-based implementation and security analysis

Abstract

This report analyses the four exercises for the Information Security Lab 4 brief: AES symmetric encryption, RSA public-key encryption, SHA-256 hashing, and RSA digital signatures. The Python project contains independent scripts and a combined runner. The full combined execution recorded returned passed=true for all exercises under Python 3.12.14 and cryptography 50.0.2. The checks demonstrate decryption equality, authenticated-ciphertext rejection, SHA-256 output sensitivity to a one-bit input change, and signature verification and rejection of a modified message. They do not establish production readiness, operational key custody, signer identity, performance, or resistance to untested attacks.

Numerical results are taken from the current execution logs. AES, RSA, SHA-256 and signature screenshots are screenshot is available and the source files code are reproduced in Appendices A-D.

Keywords: *authenticated encryption, RSA-OAEP, RSA-PSS, SHA-256, integrity, key management*

Table of contents

Abstract 1
Table of contents 2
List of figures 4
List of tables 4
1. Introduction 5
1.1 Background 5
1.2 Objectives 5
1.3 Scope and verification approach 5
1.4 Technologies 5
2. Experimental environment 5
2.1 Verified host and software configuration 5
2.2 Project organization 6
2.3 Installation and execution 6
2.4 Combined execution record. 6
3. Exercise 1 - Symmetric encryption (AES) 7
3.1 Objective 7
3.2 Theoretical background 7
3.3 Algorithm and implementation 7
3.4 Important code components 7
3.5 Experimental procedure 7
3.6 Experimental results 7
3.7 Screenshot and evidence 8
3.8 Security analysis 8
3.9 Discussion 8
4. Exercise 2 - Asymmetric encryption (RSA) 9
4.1 Objective 9
4.2 Theoretical background 9
4.3 Algorithm and implementation 9
4.4 Important code components 9
4.5 Experimental procedure 9
4.6 Experimental results 9
4.7 Screenshot and evidence 10
4.8 Security analysis 10
4.9 Discussion 10
5. Exercise 3 - Hash functions (SHA-256) 11
5.1 Objective 11
5.2 Theoretical background 11

5.3 Algorithm and implementation 11
5.4 Important code components 11
5.5 Experimental procedure 11
5.6 Experimental results 11
5.7 Screenshot and evidence 12
5.8 Security analysis 12
5.9 Discussion 12
6. Exercise 4 - Digital signatures (RSA-PSS) 13
6.1 Objective 13
6.2 Theoretical background 13
6.3 Algorithm and implementation 13
6.4 Important code components 13
6.5 Experimental procedure 13
6.6 Experimental results 13
6.7 Screenshot and evidence 14
6.8 Security analysis 14
6.9 Discussion 14
7. Comparative security analysis 15
8. Overall discussion 16
8.1 Learning outcomes 16
8.2 Practical applications 16
8.3 Limitations and reproducibility 16
8.4 Evidence status 16
9. Conclusion 17
10. References 18
Appendix A: Complete AES implementation 19
Appendix B: Complete RSA-OAEP implementation 20
Appendix C: Complete SHA-256 implementation 21
Appendix D: Complete RSA-PSS signature implementation 22

List of figures

Figure 1. Output for thr AES implementation 8
Figure 3. RSA-OAEP terminal run 10
Figure 4. SHA-256 comparison output 12
Figure 5. RSA-PSS signing and verification 14

List of tables

Table 1. *Verified execution environment* 5
Table 2. *Combined run outcomes* 6
Table 3. *AES combined-run values* 7
Table 4. *RSA-OAEP combined-run observations* 9
Table 5. *SHA-256 inputs and observed digests* 11
Table 6. *RSA-PSS signature observations* 13
Table 7. *Comparison of laboratory mechanisms* 15

1. Introduction

1.1 Background

Cryptography supplies mechanisms for confidentiality, integrity, entity authentication, and evidence of message origin. These properties arise from distinct constructions and key-management assumptions. AES is a symmetric block cipher; a mode such as GCM turns it into authenticated encryption. RSA supports separate encryption and signature schemes. SHA-256 maps arbitrary byte strings to a fixed 256-bit digest. A digest can reveal changes only when a trusted reference value is available; it does not authenticate a sender or conceal data [1]-[8].

1.2 Objectives

- Implement and run the four laboratory exercises with the Python cryptography package.
- Observe encryption/decryption, digest, and signature behavior on the actual sample inputs.
- Compare AES key sizes and modes and RSA key sizes as requested by the lab variations.
- Interpret measured outputs while separating experimental evidence from general security claims.

1.3 Scope and verification approach

The review covers the four source programs, README, requirements, combined runner, per-exercise logs, available screenshots, and official two-page lab brief. The combined runner was executed and its timestamped JSON output supplies the report's numeric values. The standalone AES screenshot is from a distinct earlier run and is used only to demonstrate the workflow; its random nonce and ciphertext are not attributed to the combined run.

These are demonstrations on short sample inputs, not a cryptographic certification, formal proof, comprehensive attack assessment, or deployed-system security evaluation. No performance benchmark, large-file encryption, key lifecycle, cross-process nonce coordination, or key-compromise recovery experiment was run.

1.4 Technologies

The recorded environment is Python 3.12.14, cryptography 50.0.2, Visual Studio Code 1.144.0, and Windows 11 Pro Insider Preview build 28120. The host configuration is listed in Section 2. The official lab brief defines the exercise objectives; technical standards and API documentation are cited in Section 10.

2. Experimental environment

2.1 Verified host and software configuration

Table 1. *Verified execution environment*

Component	Observed configuration
Operating system	Windows 11 Pro Insider Preview, version 10.0.28120, build 28120
Processor	Intel Core i5-10310U at 1.70 GHz; 8 logical processors
Installed memory	11.7 GB physical memory reported by Windows
Python runtime	3.12.14 (project virtual environment)
cryptography	50.0.2
Editor	Visual Studio Code 1.144.0
Dependency declaration	requirements.txt: cryptography>=42.0.0
Project path	C:\Users\MrRobot\Documents\Lab4-Cryptography

2.2 Project organization

The four scripts are separated by exercise. run_all.py imports each run () function and writes experiment_log. json plus per-exercise logs under screenshots/. The included venv is local to this host and excluded by .gitignore; recreate it from requirements.txt on another computer.

- exercise1_aes/aes_demo.py - AES-GCM and AES-CBC/PKCS7 comparison.
- exercise2_rsa/rsa_demo.py - RSA-OAEP encryption/decryption and key-size comparison.
- exercise3_sha256/sha256_demo.py - SHA-256 on text, binary bytes, and a JPEG.
- exercise4_signatures/signature_demo.py - RSA-PSS signing, verification, and changed-message rejection.
- screenshots/ - JSON/text experiment logs and available screenshots.
- run_all.py - combined execution, runtime metadata, and per-exercise log writer.

2.3 Installation and execution

From the project root in PowerShell, recreate and activate the virtual environment, install the dependency, and execute the combined experiment:

```
cd C:\Users\MrRobot\Documents\Lab4-Cryptography
.\venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python .\run_all.py
```

For an individual run from the project root, use python .\exercise1_aes\aes_demo.py, python .\exercise2_rsa\rsa_demo.py, python .\exercise3_sha256\sha256_demo.py, or python .\exercise4_signatures\signature_demo.py. From inside an exercise folder, use python .\aes_demo.py (or the corresponding filename); PowerShell does not execute a current-folder script by its bare name.

2.4 Combined execution record.

The combined run reported passed=true for each exercise. These flags are the code's own checks, not an independent certification of cryptographic strength.

Table 2. *Combined run outcomes*

Exercise	Configuration	Status	Recorded checks
AES encryption	AES-256-GCM	PASS	Decrypt equality; GCM tag rejection; key-size/CBC round trips
RSA encryption	RSA-2048-OAEP-SHA-256	PASS	OAEP decrypt equality; modified ciphertext rejected; key-size rounds
Hashing	SHA-256	PASS	256-bit output; changed input; binary/image hashes
Digital signature	RSA-2048-PSS-SHA-256	PASS	Original verifies; modified message rejected

The full output is in screenshots/experiment_output.txt and exact results are in experiment_log.json and the per-exercise JSON files.

3. Exercise 1 - Symmetric encryption (AES)

3.1 Objective

Implement AES encryption and decryption, observe key-size and mode variations, and analyze confidentiality and integrity.

3.2 Theoretical background

AES is a 128-bit block cipher with 128-, 192-, or 256-bit keys [1]. A mode is required to process arbitrary-length data. The primary implementation uses AES-GCM: counter-mode encryption is combined with authentication and optional additional authenticated data (AAD). AAD is authenticated but remains visible. AESGCM appends a 128-bit tag to the ciphertext and verifies it before returning plaintext [2], [3].

3.3 Algorithm and implementation

The program generates a fresh 256-bit key and a random 12-byte (96-bit) nonce, supplies AAD course=information-security;exercise=AES-GCM, encrypts a fixed UTF-8 message, and decrypts with the same context. It flips one ciphertext bit and expects InvalidTag. Separate comparisons perform AES-128-GCM, AES-256-GCM, and AES-256-CBC with PKCS7 padding.

3.4 Important code components

- AESGCM.generate_key(bit_length=256) generates the key; the key is kept in memory and omitted from output.
- os.urandom(12) produces the 96-bit nonce. A one-use key avoids reuse within this demonstration; persistent keys require a uniqueness strategy across runs and processes.
- encrypt(nonce, plaintext, associated_data) protects the plaintext and authenticates ciphertext plus AAD; decrypt receives the same AAD.
- The InvalidTag check demonstrates rejection of the altered ciphertext, not resistance to all operational failures.
- CBC/PKCS7 is an educational comparison. The code explicitly records authenticated=false because bare CBC does not authenticate data.

3.5 Experimental procedure

- Run the script using the project environment and fixed plaintext.
- Record the key size, nonce, AAD, ciphertext/tag, recovered plaintext, and checks without logging the secret.
- Compare AES-128-GCM and AES-256-GCM round trips, then AES-256-CBC/PKCS7.
- Modify one ciphertext bit and verify GCM authentication rejects it.

3.6 Experimental results

Table 3. *AES combined-run values*

Measure	Observed result
Primary construction	AES-256-GCM
Key and nonce	256 bits; 96-bit nonce c2b632fa1f2d6697e1b9b18b
AAD	course=information-security;exercise=AES-GCM
Plaintext	Lab 4: protect this confidential message.
Recovered plaintext	Lab 4: protect this confidential message.
Ciphertext + tag	57 bytes; exact hex in aes_log.json
GCM checks	decryption matches=true; modified ciphertext rejected=true
Key-size variations	AES-128-GCM and AES-256-GCM both round-tripped; 57 bytes each including tag

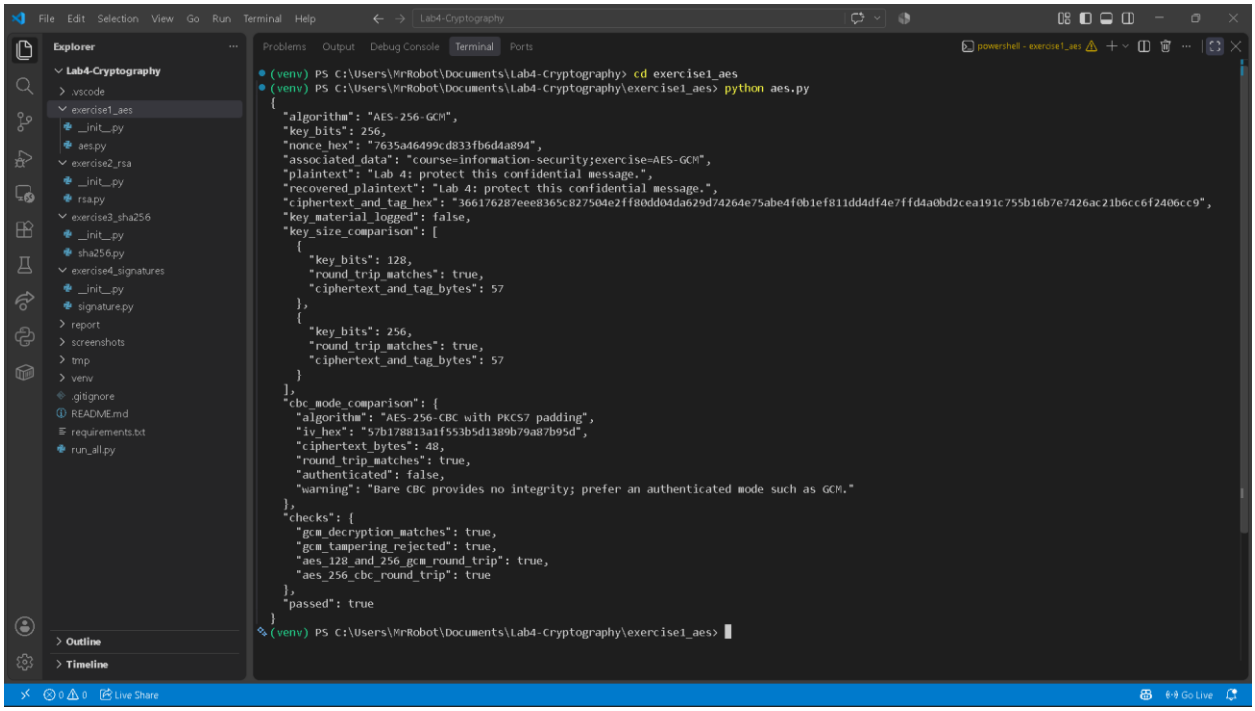
Measure	Observed result
CBC variation	round-trip=true; 48 ciphertext bytes; authenticated=false
Overall	PASS

The plaintext and recovered plaintext match. For the fixed message and tag length, the AES-128 and AES-256 GCM outputs are both 57 bytes; key size is not encoded by output length. The CBC path also round-tripped unmodified data, but it provides no integrity.

3.7 Screenshot and evidence

Figure 1 is an actual standalone AES terminal capture. Its random nonce and ciphertext belong to that separate run; the result table uses the combined-run log timestamp above. The standalone JSON output is retained in screenshots/aes_execution.json.

Figure 1. Output for thr AES implementation



3.8 Security analysis

GCM protects confidentiality and integrity of ciphertext and authenticates AAD, assuming correct key handling, a valid tag, and nonce uniqueness per key. NIST recommends a 96-bit IV length for GCM [2]; the cryptography API warns that nonce reuse with a key compromises security [3]. This demonstration uses a fresh one-use key, but a persistent key requires uniqueness across restarts and processes. AAD is not encrypted. The key is not persisted, wrapped, rotated, or recovered.

CBC here supplies confidentiality only. It can yield unauthenticated output after modification, and exposed padding errors can create padding-oracle risk. A production CBC construction must add sound authentication and careful error handling; AEAD is generally preferred for new protocols. The CBC test did not modify ciphertext.

3.9 Discussion

The experiment separates encryption from authentication: both modes recover unchanged data, but GCM additionally rejects a tampered ciphertext. The key-size result is functional, not a performance benchmark or proof that AES-256 is always preferable to AES-128. Serialization, key distribution, replay protection, and secure storage are outside this implementation.

4. Exercise 2 - Asymmetric encryption (RSA)

4.1 Objective

Generate a public/private pair, encrypt with the public key, decrypt with the private key, and observe modulus-size and ciphertext-modification behavior.

4.2 Theoretical background

RSA is a public-key algorithm. RSA-OAEP is the randomized encryption encoding specified by PKCS #1; this implementation uses SHA-256 for OAEP and MGF1 [4], [5]. Anyone with the authentic public key can encrypt to its owner, while the matching private key decrypts. Encryption alone does not authenticate the sender. RSA's input capacity is limited by modulus size and padding, so it is suited here to a short secret, not bulk data.

4.3 Algorithm and implementation

The script generates an RSA-2048 key pair with exponent 65537, exports the public key as DER SubjectPublicKeyInfo to compute a SHA-256 fingerprint, configures OAEP/SHA-256, encrypts a short message, and decrypts it. It flips a ciphertext byte and treats ValueError as rejection. A comparison repeats round trips with 2048-bit and 3072-bit moduli.

4.4 Important code components

- generate_private_key(public_exponent=65537, key_size=2048) creates an ephemeral private key; public_key() derives its public counterpart.
- OAEP uses SHA-256 and MGF1(SHA-256), with no label.
- public_key.encrypt(...) and private_key.decrypt(...) demonstrate the key direction for confidentiality.
- The modified ciphertext test confirms rejection for this sample; it does not assess timing or protocol-safe error behavior.
- The public-key fingerprint identifies the key bytes but does not bind them to a verified person or organization.

4.5 Experimental procedure

- Generate the RSA-2048 key pair and OAEP parameters.
- Encrypt the fixed short message with the public key and decrypt with the private key.
- Modify one ciphertext byte and observe rejection.
- Repeat round trips at 2048 and 3072 bits and compare lengths.

4.6 Experimental results

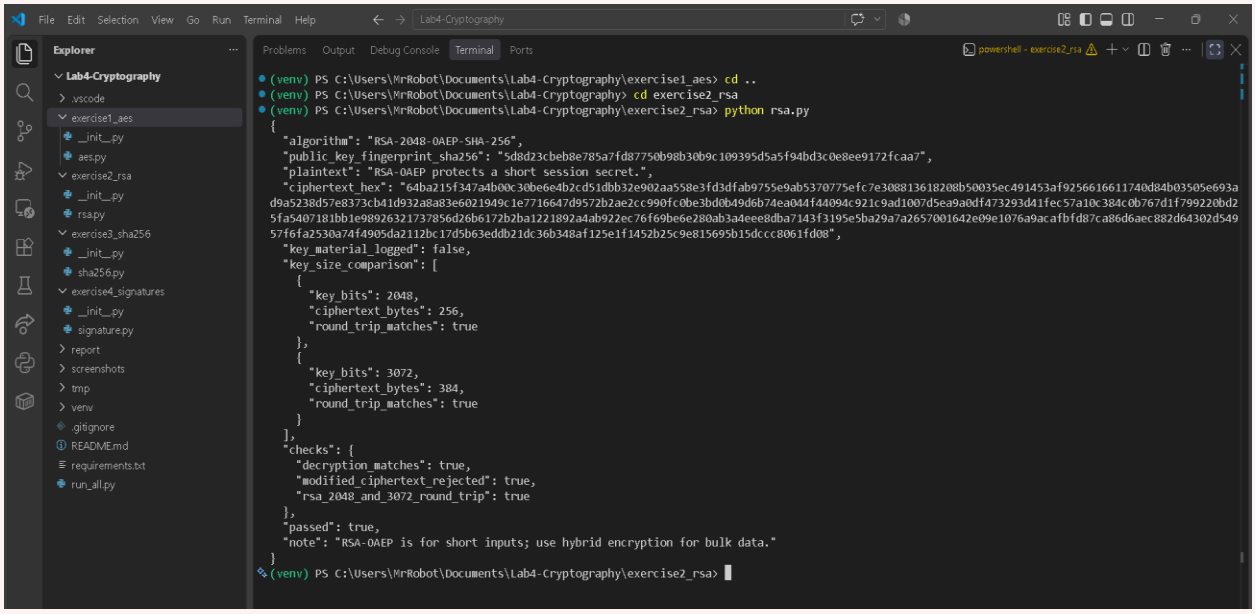
Table 4. ***RSA-OAEP combined-run observations***

Measure	Observed result
Primary algorithm	RSA-2048-OAEP-SHA-256
Recovered plaintext	RSA-OAEP protects a short session secret.
Size comparison	2048-bit: 256 bytes, match=True; 3072-bit: 384 bytes, match=True
Modified ciphertext	Rejected
Public-key fingerprint	3a7bcc96fd4366dec2d39cf7f2e071e1e27482779c626c4d151c322b749a4e37
Overall	PASS

RSA-2048 produced a 256-byte ciphertext and RSA-3072 a 384-byte ciphertext, matching modulus lengths. These functional observations do not measure computation time or quantify security strength.

4.7 Screenshot and evidence

Figure 2. RSA-OAEP terminal run



No dedicated RSA screenshot is present. The current per-exercise JSON log and combined stdout record the results above.

4.8 Security analysis

OAEP is the appropriate RSA encryption padding for new use in this API. Safe deployment also requires authenticating the recipient public key, protecting the private key, and avoiding distinguishable decryption errors that can expose an oracle. The private key here is generated in memory and is neither persisted nor protected by a hardware module or passphrase. The logs contain public fingerprints and ciphertext, not private parameters.

RSA-OAEP provides confidentiality to the matching private-key holder under its assumptions. It does not by itself authenticate the sender, provide protocol replay protection, or establish non-repudiation. Bulk data should use a reviewed hybrid-encryption construction [4], [5].

4.9 Discussion

The original message was recovered and the altered ciphertext rejected. Fresh keys and randomized OAEP mean fingerprints and ciphertexts vary across runs; reports should identify the run log rather than compare ciphertext bytes with a different screenshot.

5. Exercise 3 - Hash functions (SHA-256)

5.1 Objective

Hash different input types, compare outputs, explain the observed avalanche effect, and distinguish integrity checking from authentication and secrecy.

5.2 Theoretical background

SHA-256 is part of the SHA-2 family in NIST's Secure Hash Standard and returns a 256-bit digest [6], [7]. A hash is deterministic and non-reversible. A trusted digest can help detect changed bytes, but an unkeyed digest does not authenticate a sender if an attacker can replace both file and digest. A MAC or signature is required when authenticity is needed.

5.3 Algorithm and implementation

The script hashes the empty byte string, a text sample, a version with uppercase C changed to lowercase c (one ASCII bit flips), five arbitrary bytes, and the exact JPEG file screenshots/vscode_run.jpg. It XORs the two text digests and counts set bits to report their Hamming distance.

5.4 Important code components

- hashlib.sha256(...).hexdigest() renders a 256-bit digest as 64 hexadecimal characters.
- The binary sample is hashed as bytes without decoding.
- Path(__file__).resolve().parents[1] locates the image relative to the script.
- XOR plus bit_count() measures this pair's digest distance; it is not a statistical study of avalanche behavior.

5.5 Experimental procedure

- Hash the empty input and two text samples with one input bit changed.
- Hash the fixed binary bytes and JPEG screenshot file.
- Compare the outputs and count differing output bits for the text pair.

5.6 Experimental results

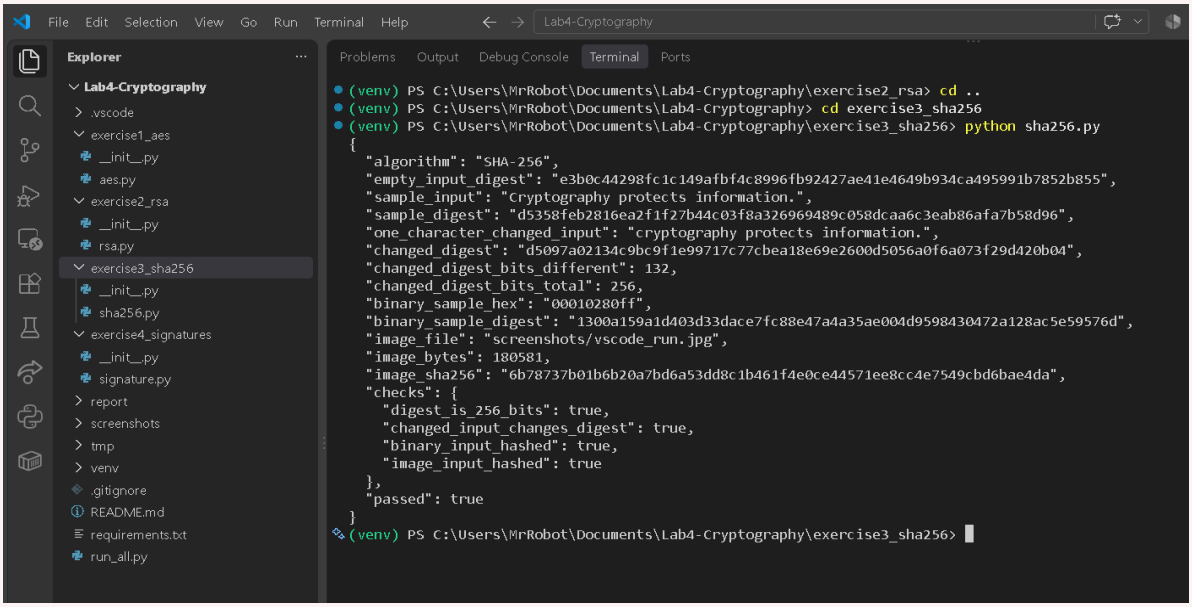
Table 5. *SHA-256 inputs and observed digests*

Input/test	Observed result
Empty input	e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855
Sample text	Cryptography protects information. -> d5358feb2816ea2f1f27b44c03f8a326969489c058dcaa6c3eab86afa7b58d96
Changed text	cryptography protects information. -> d5097a02134c9bc9f1e99717c77cbea18e69e2600d5056a0f6a073f29d420b04
Digest distance	132 of 256 bits
Binary bytes	00010280ff -> 1300a159a1d403d33dace7fc88e47a4a35ae004d9598430472a128ac5e59576d
JPEG	screenshots/vscode_run.jpg; 180581 bytes; SHA-256 6b78737b01b6b20a7bd6a53dd8c1b461f4e0ce44571ee8cc4e7549cbd6bae4da
Overall	PASS

The one-bit input change produced a different digest and changed 132 of 256 digest bits in this sample, consistent with the avalanche effect. One observation cannot prove the expected distribution or collision resistance. The code does not attempt to find collisions; collision resistance is a cryptographic design property assessed through analysis, not inferred from a small sample.

5.7 Screenshot and evidence

Figure 3. SHA-256 comparison output



5.8 Security analysis

SHA-256 is suitable for an integrity fingerprint when the expected value comes from a trusted channel. It is not a password-storage function; a salted, deliberately expensive password-hashing scheme is required. It does not encrypt or authenticate the sample. The experiment verifies computation and one input-sensitivity case, not cryptanalytic collision or preimage resistance.

5.9 Discussion

Text, binary, and image samples show that hashing operates on bytes independent of text structure. The empty input matches the known SHA-256 empty-string digest. A deployment should bind hashes to a trusted manifest, MAC, signature, or authenticated protocol according to its objective.

6. Exercise 4 - Digital signatures (RSA-PSS)

6.1 Objective

Generate an RSA key pair, sign with RSA-PSS/SHA-256, verify the original message, and reject verification after the message changes.

6.2 Theoretical background

A signature is produced with a private key and verified with its public counterpart. A valid signature supports integrity and possession of the signing key; signer identity requires a trusted binding between the public key and the claimed signer. It does not encrypt the message. RSA-PSS is a probabilistic RSA signature encoding specified in PKCS #1 and is used with SHA-256 here [4], [8]. Non-repudiation claims additionally depend on key custody, identity assurance, policy, and dispute context.

6.3 Algorithm and implementation

The script creates a separate RSA-2048 key pair with exponent 65537, hashes the DER public key to produce a fingerprint, configures PSS with MGF1(SHA-256) and maximum salt length, signs a fixed message, and verifies it. It appends the bytes (changed) and expects InvalidSignature.

6.4 Important code components

- private_key.sign(message, pss, hashes.SHA256()) creates the signature; the public key verifies.
- PSS uses a salt, so signature bytes may differ between executions.
- The negative test changes the message and treats InvalidSignature as expected.
- The fingerprint identifies demonstration key bytes; no certificate or trusted signer directory is present.

6.5 Experimental procedure

- Generate a separate signing pair and public-key fingerprint.
- Sign the original message and verify it with the public key.
- Change the message and confirm verification fails.

6.6 Experimental results

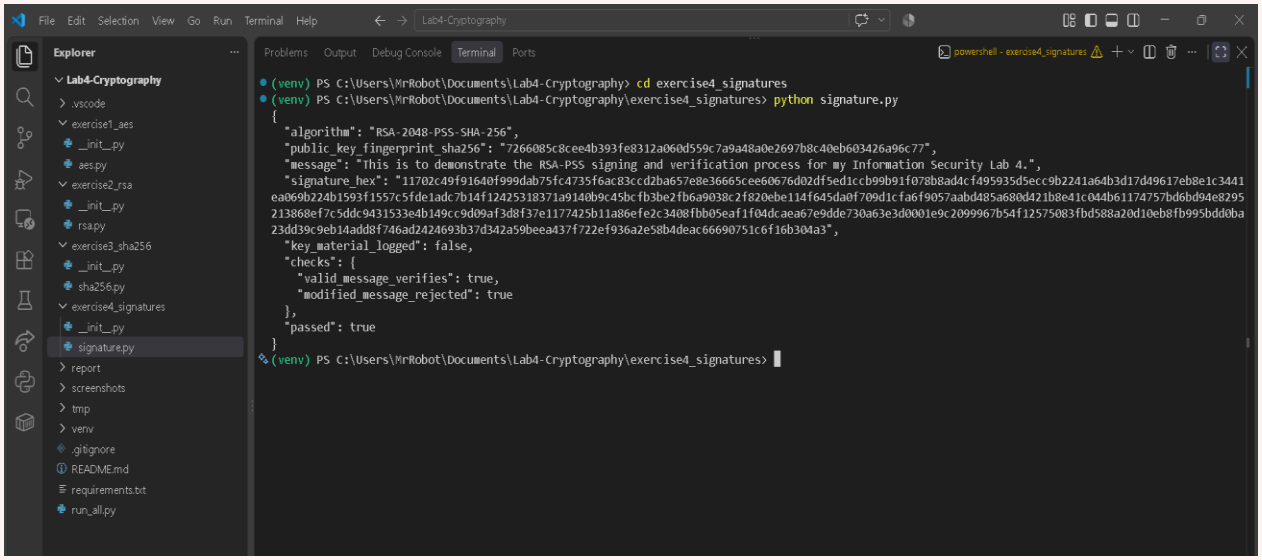
Table 6. ***RSA-PSS signature observations***

Measure	Observed result
Algorithm	RSA-2048-PSS-SHA-256
Message	This message was approved for the lab demonstration.
Original message	Verified
Modified message	Rejected
Signature length	256 bytes
Public-key fingerprint	23260f3897a5fecb14dffe99938ef4f3511d11a86dae589d10a4bfe68132bceb
Overall	PASS

The original verified and the modified message was rejected. The 2048-bit RSA signature is 256 bytes. The result is about the ephemeral key pair in the logged run, not the identity of a real-world signer.

6.7 Screenshot and evidence

Figure 4. RSA-PSS signing and verification



No dedicated signature screenshot is present. The per-exercise and combined logs record both checks and the public-key fingerprint; no private key material is logged.

6.8 Security analysis

RSA-PSS/SHA-256 is a standards-backed signature construction. Verification detects changes and establishes possession of the corresponding private key, subject to cryptographic assumptions. Authentication of a person or organization requires trusted public-key binding and lifecycle management. The ephemeral key is not tied to a verified identity, so the experiment does not establish legal or operational non-repudiation. Key compromise, revocation, and signing authorization are outside scope.

6.9 Discussion

The exercise demonstrates verification independently of RSA encryption and uses a separate key pair. It does not encrypt the message; the signature is public. A production signing workflow needs access control, key protection, identity validation, policy, and lifecycle management.

7. Comparative security analysis

The primitives address different objectives and are not interchangeable. Table 6 summarizes their intended properties and the boundary of this implementation.

Table 7. ***Comparison of laboratory mechanisms***

Mechanism	Type	Security property	Strength	Limitation
AES-256-GCM	Symmetric AEAD	Confidentiality; ciphertext/AAD integrity	Efficient bulk protection; authenticates AAD	Secret-key custody and nonce uniqueness; no sender identity
AES-CBC/PKCS7	Symmetric mode	Confidentiality only here	Demonstrates padding and chaining	No authentication; malleability and padding-oracle exposure
RSA-OAEP/SHA-256	Public-key encryption	Confidentiality to private-key holder	Public key distribution; randomized encoding	Short input limit; private-key custody; no sender authentication
SHA-256	Unkeyed hash	Change detection only with trusted reference	Fixed digest for arbitrary bytes	No secrecy/authenticity; not password hashing; no collision test
RSA-PSS/SHA-256	Public-key signature	Integrity and possession of signing key	Public verification; changed-message rejection	Private key protection and identity binding; no encryption

A typical hybrid design protects bulk data with an AEAD key and uses an approved key-establishment or key-wrapping mechanism for that key. A signature can separately authenticate the document when the verifier trusts the signing key. SHA-256 alone does neither encryption nor authenticity. Any composition needs protocol-specific analysis; these stand-alone scripts are not a complete secure-messaging protocol.

8. Overall discussion

8.1 Learning outcomes

- Symmetric and public-key encryption recover plaintext but depend on different key relationships and operating constraints.
- Authenticated encryption combines confidentiality and integrity; altered ciphertext must be rejected before plaintext is trusted.
- A plain digest detects change only when the reference digest is trusted.
- A signature supports integrity and signing-key possession relative to a trusted public key; it does not encrypt content or independently identify a person.
- Nonces, padding, AAD, key protection, and public-key trust are core system-design concerns.

8.2 Practical applications

AES-GCM is useful for authenticated application records, secure transport payloads, and storage encryption when keys and nonces are managed correctly. RSA-OAEP can protect short secrets in compatible systems. SHA-256 is used for file fingerprints and as a signature component. RSA-PSS can sign software releases and documents when the verifier validates key identity and status. These scripts demonstrate the primitives, not the surrounding protocols.

8.3 Limitations and reproducibility

Fresh random keys, nonces, OAEP encodings, and PSS salts make bytes vary between executions. Reproducibility means preserving the timestamp and log rather than expecting identical ciphertext or signatures. The image hash is tied to the exact JPEG bytes. The AES screenshot is from a stand-alone execution, separate from the combined result table.

No protocol integration, certificates, persistent key storage, hardware security module, rotation/revocation, backup, side-channel analysis, concurrency, timing, or post-quantum migration was evaluated. The experiments are short and do not test large files or benchmark performance.

8.4 Evidence status

All four scripts passed in the combined logged run. A dedicated AES screenshot is available. Dedicated RSA, SHA-256, and RSA-PSS screenshots are missing; their per-exercise JSON logs are present. The official lab permits screenshots or logs as experiment evidence. Appendix E and report/Evidence_Inventory.md describe the files, provenance, and exact recommended captures.

9. Conclusion

The project implements and executes all four objectives in the official Lab 4 brief. AES-256-GCM recovered the plaintext and rejected altered ciphertext; the run also compared AES key sizes and an unauthenticated CBC mode. RSA-OAEP recovered its short sample, rejected modified ciphertext, and produced modulus-sized ciphertexts for 2048- and 3072-bit keys. SHA-256 generated 256-bit digests for text, binary, and JPEG input; the one-bit text change differed in 132 of 256 digest bits. RSA-PSS verified the original message and rejected its modification.

All four self-checks returned passed=true. The evidence does not establish production key management, real-world public-key identity, performance, general collision resistance, or legal non-repudiation. Machine-readable logs cover all exercises, one current AES screenshot is embedded, and three missing screenshots are explicitly marked with capture instructions. The report therefore separates completed experiments from outstanding visual evidence.

10. References

[1] NIST. Advanced Encryption Standard (AES), FIPS PUB 197, updated May 2023. https://csrc.nist.gov/pubs/fips/197/final

[2] NIST. Recommendation for Block Cipher Modes: GCM and GMAC, SP 800-38D, 2007. https://csrc.nist.gov/pubs/sp/800/38/d/final

[3] Python Cryptographic Authority. cryptography documentation: Authenticated encryption (AESGCM). Accessed 2 November 2026. https://cryptography.io/en/latest/hazmat/primitives/aead/

[4] Moriarty, K., et al. PKCS #1: RSA Cryptography Specifications Version 2.2, RFC 8017, IETF, 2016. https://www.rfc-editor.org/rfc/rfc8017

[5] Python Cryptographic Authority. cryptography documentation: RSA. Accessed 2 November 2026. https://cryptography.io/en/latest/hazmat/primitives/asymmetric/rsa/

[6] NIST. Secure Hash Standard (SHS), FIPS PUB 180-4, 2015. https://csrc.nist.gov/pubs/fips/180-4/upd1/final

[7] Python Cryptographic Authority. cryptography documentation: Message digests (hashing). Accessed 2 November 2026. https://cryptography.io/en/latest/hazmat/primitives/cryptographic-hashes/

[8] NIST. Digital Signature Standard (DSS), FIPS PUB 186-5, 2023. https://csrc.nist.gov/pubs/fips/186-5/final

[9] ESIGELEC. Information Security, Lab 4: Cryptography Fundamentals. Supplied course laboratory brief, 2 pages.

Appendix A: Complete AES implementation

Complete source file: exercise1_aes/aes_demo.py

```
"""Exercise 1: compare AES-GCM and AES-CBC encryption modes."""

import json
import os

from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives import padding as symmetric padding
from cryptography.hazmat.primitives.ciphers import Cipher, algorithms, modes
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


def run() -> dict:
    plaintext = b"Lab 4: protect this confidential message."
    associated data = b"course=information-security;exercise=AES-GCM"

    key = AESGCM.generate key(bit length=256)
    nonce = os.urandom(12)
    cipher = AESGCM(key)
    ciphertext = cipher.encrypt(nonce, plaintext, associated data)
    recovered = cipher.decrypt(nonce, ciphertext, associated data)

    changed = bytearray(ciphertext)
    changed[0] ^= 1
    tampering_rejected = False
    try:
        cipher.decrypt(nonce, bytes(changed), associated data)
    except InvalidTag:
        tampering rejected = True

    key size comparison = []
    for bits in (128, 256):
        comparison key = AESGCM.generate key(bit length=bits)
        comparison nonce = os.urandom(12)
        comparison cipher = AESGCM(comparison key)
        comparison_ct = comparison_cipher.encrypt(comparison_nonce, plaintext, associated_data)
        comparison pt = comparison cipher.decrypt(comparison nonce, comparison ct, associated data)
        key size comparison.append({
            "key bits": bits,
            "round trip matches": comparison pt == plaintext,
            "ciphertext and tag bytes": len(comparison ct),
        })

    # Educational mode comparison only: bare CBC does not authenticate data.
    cbc key = os.urandom(32)
    iv = os.urandom(16)
    padder = symmetric padding.PKCS7(algorithms.AES.block size).padder()
    padded plaintext = padder.update(plaintext) + padder.finalize()
    encryptor = Cipher(algorithms.AES(cbc key), modes.CBC(iv)).encryptor()
    cbc ciphertext = encryptor.update(padded plaintext) + encryptor.finalize()
    decryptor = Cipher(algorithms.AES(cbc key), modes.CBC(iv)).decryptor()
    padded recovered = decryptor.update(cbc ciphertext) + decryptor.finalize()
    unpadder = symmetric padding.PKCS7(algorithms.AES.block size).unpadder()
    cbc recovered = unpadder.update(padded recovered) + unpadder.finalize()

    checks = {
        "gcm decryption matches": recovered == plaintext,
        "gcm tampering rejected": tampering rejected,
        "aes 128 and 256 gcm round trip": all(x["round trip matches"] for x in key size comparison),
        "aes 256 cbc round trip": cbc recovered == plaintext,
    }
    return {
        "algorithm": "AES-256-GCM",
        "key bits": len(key) * 8,
        "nonce hex": nonce.hex(),
        "associated data": associated data.decode("ascii"),
        "plaintext": plaintext.decode("utf-8"),
        "recovered plaintext": recovered.decode("utf-8"),
        "ciphertext and tag hex": ciphertext.hex(),
        "key material logged": False,
        "key size comparison": key size comparison,
        "cbc mode comparison": {
            "algorithm": "AES-256-CBC with PKCS7 padding",
            "iv hex": iv.hex(),
            "ciphertext bytes": len(cbc ciphertext),
            "round trip matches": cbc recovered == plaintext,
            "authenticated": False,
            "warning": "Bare CBC provides no integrity; prefer an authenticated mode such as GCM.",
        },
        "checks": checks,
        "passed": all(checks.values()),
    }


if   name   == "  main  ":
    print(json.dumps(run(), indent=2))
```

Appendix B: Complete RSA-OAEP implementation

Complete source file: exercise2_rsa/rsa_demo.py

```
"""Exercise 2: RSA-OAEP encryption and key-size comparison."""

import hashlib
import json

from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa


def run() -> dict:
    private key = rsa.generate private key(public exponent=65537, key size=2048)
    public key = private key.public key()
    public der = public key.public bytes(
        encoding=serialization.Encoding.DER,
        format=serialization.PublicFormat.SubjectPublicKeyInfo,
    )
    fingerprint = hashlib.sha256(public der).hexdigest()
    oaep = padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None,
    )
    plaintext = b"RSA-OAEP protects a short session secret."
    ciphertext = public_key.encrypt(plaintext, oaep)
    recovered = private key.decrypt(ciphertext, oaep)

    changed = bytearray(ciphertext)
    changed[-1] ^= 1
    tampering rejected = False
    try:
        private key.decrypt(bytes(changed), oaep)
    except ValueError:
        tampering rejected = True

    size_comparison = []
    for bits in (2048, 3072):
        comparison private = rsa.generate private key(public exponent=65537, key size=bits)
        comparison public = comparison private.public key()
        comparison ciphertext = comparison public.encrypt(plaintext, oaep)
        comparison plaintext = comparison private.decrypt(comparison ciphertext, oaep)
        size comparison.append({
            "key_bits": bits,
            "ciphertext bytes": len(comparison ciphertext),
            "round trip matches": comparison plaintext == plaintext,
        })

    checks = {
        "decryption matches": recovered == plaintext,
        "modified ciphertext rejected": tampering rejected,
        "rsa 2048 and 3072 round trip": all(x["round trip matches"] for x in size comparison),
    }
    return {
        "algorithm": "RSA-2048-OAEP-SHA-256",
        "public key fingerprint sha256": fingerprint,
        "plaintext": plaintext.decode("utf-8"),
        "ciphertext hex": ciphertext.hex(),
        "key material logged": False,
        "key size comparison": size comparison,
        "checks": checks,
        "passed": all(checks.values()),
        "note": "RSA-OAEP is for short inputs; use hybrid encryption for bulk data.",
    }


if   name   == "  main  ":
    print(json.dumps(run(), indent=2))
```

Appendix C: Complete SHA-256 implementation

Complete source file: exercise3_sha256/sha256_demo.py

```
"""Exercise 3: SHA-256 for text, binary bytes, and an image file."""

import hashlib
import json
from pathlib import Path


def run() -> dict:
    sample = b"Cryptography protects information."
    changed = b"cryptography protects information."
    empty digest = hashlib.sha256(b"").hexdigest()
    sample digest = hashlib.sha256(sample).hexdigest()
    changed digest = hashlib.sha256(changed).hexdigest()
    differing bits = (int(sample digest, 16) ^ int(changed digest, 16)).bit count()

    binary sample = bytes([0, 1, 2, 128, 255])
    binary digest = hashlib.sha256(binary sample).hexdigest()
    image path = Path(  file  ).resolve().parents[1] / "screenshots" / "vscode run.jpg"
    image bytes = image path.read bytes() if image path.is file() else b""
    image digest = hashlib.sha256(image bytes).hexdigest() if image bytes else None

    checks = {
        "digest is 256 bits": len(bytes.fromhex(sample digest)) * 8 == 256,
        "changed_input_changes_digest": sample_digest != changed_digest,
        "binary input hashed": len(bytes.fromhex(binary digest)) == 32,
        "image input hashed": image digest is not None,
    }
    return {
        "algorithm": "SHA-256",
        "empty input digest": empty digest,
        "sample input": sample.decode("utf-8"),
        "sample digest": sample digest,
        "one character changed input": changed.decode("utf-8"),
        "changed digest": changed digest,
        "changed_digest_bits_different": differing_bits,
        "changed digest bits total": 256,
        "binary sample hex": binary sample.hex(),
        "binary sample digest": binary digest,
        "image file": "screenshots/vscode run.jpg",
        "image bytes": len(image bytes),
        "image sha256": image digest,
        "checks": checks,
        "passed": all(checks.values()),
    }


if   name   == "  main  ":
    print(json.dumps(run(), indent=2))
```

Appendix D: Complete RSA-PSS signature implementation

Complete source file: exercise4_signatures/signature_demo.py

```
"""Exercise 4: RSA-PSS signing, verification, and changed-message rejection."""

import hashlib
import json

from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding, rsa

def run() -> dict:
    private key = rsa.generate private key(public exponent=65537, key size=2048)
    public key = private key.public key()
    public der = public key.public bytes(
        encoding=serialization.Encoding.DER,
        format=serialization.PublicFormat.SubjectPublicKeyInfo,
    )
    fingerprint = hashlib.sha256(public der).hexdigest()
    pss = padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt length=padding.PSS.MAX LENGTH,
    )
    message = b"This is to demonstrate the RSA-PSS signing and verification process for my Information
Security Lab 4."
    signature = private_key.sign(message, pss, hashes.SHA256())
    public key.verify(signature, message, pss, hashes.SHA256())

    changed message rejected = False
    try:
        public key.verify(signature, message + b" (changed)", pss, hashes.SHA256())
    except InvalidSignature:
        changed message rejected = True

    checks = {
        "valid message verifies": True,
        "modified_message_rejected": changed_message_rejected,
    }
    return {
        "algorithm": "RSA-2048-PSS-SHA-256",
        "public key fingerprint sha256": fingerprint,
        "message": message.decode("utf-8"),
        "signature hex": signature.hex(),
        "key_material_logged": False,
        "checks": checks,
        "passed": all(checks.values()),
    }

if   name   == "  main  ":
    print(json.dumps(run(), indent=2))
```